AWT

- AWT (Abstract Window Toolkit) is Java’s original platform-dependent GUI toolkit.
- It provides basic UI components like buttons, text fields, labels, and event handling mechanisms.

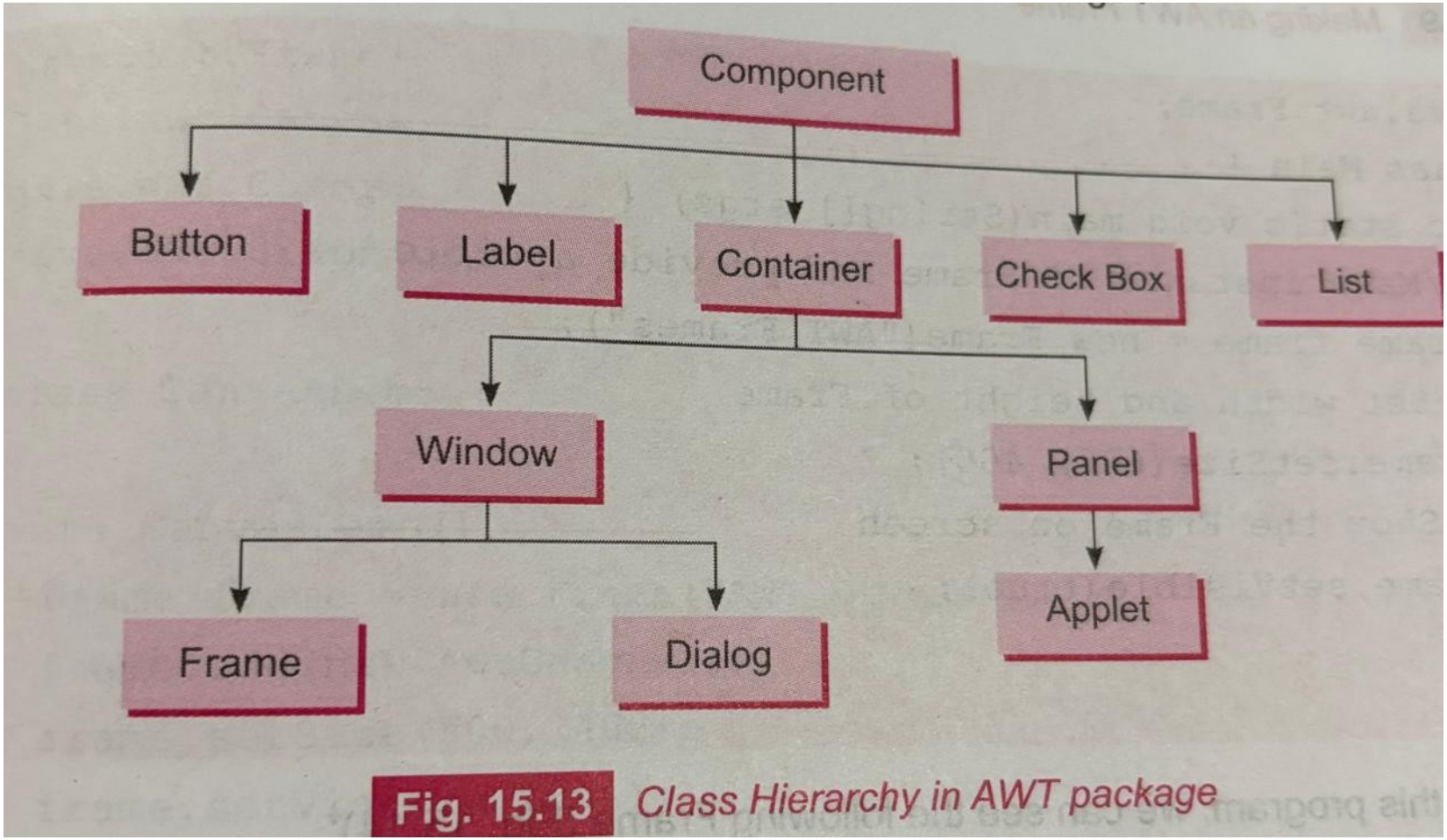
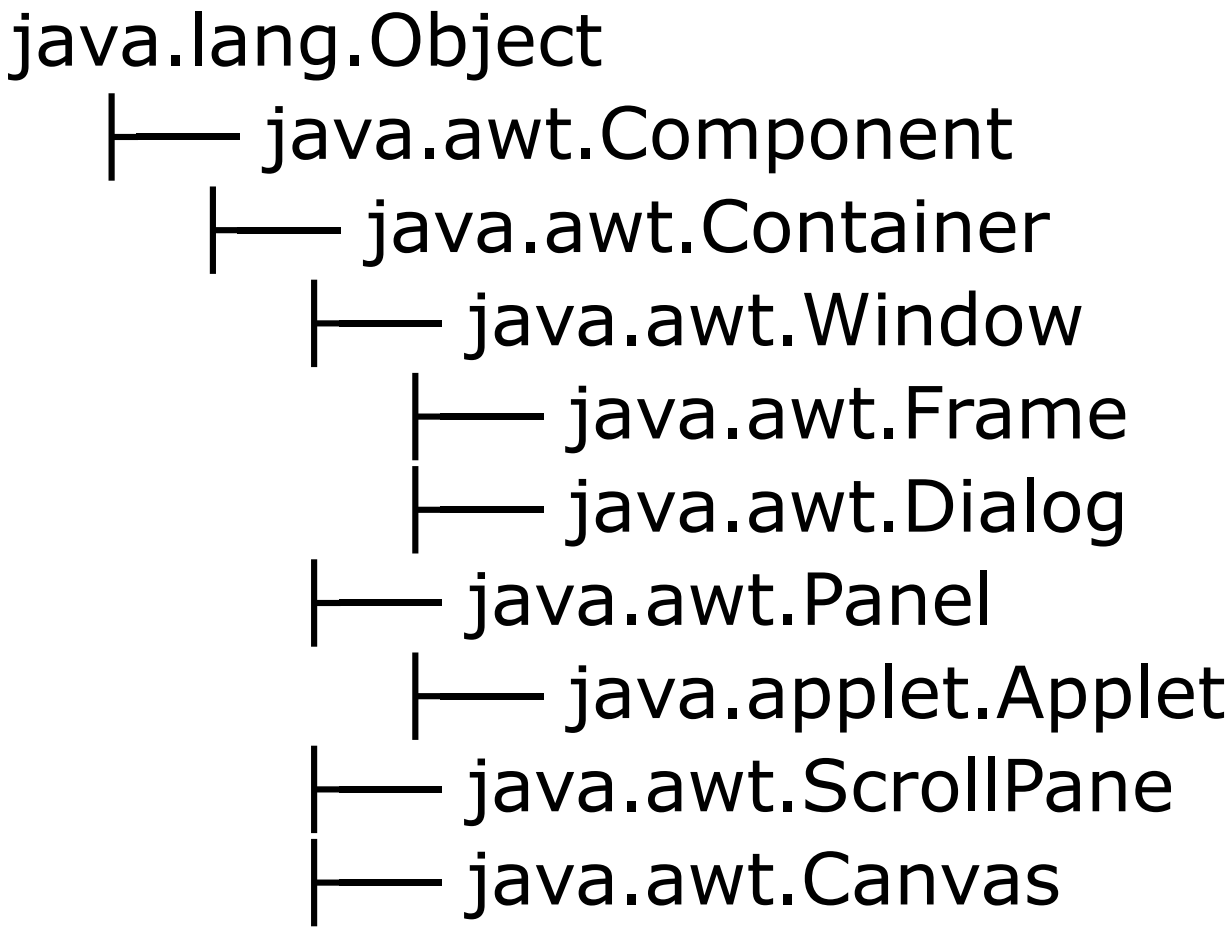


Fig. 15.13 *Class Hierarchy in AWT package*


java.lang.Object
java.awt.Component
java.awt.Container
java.awt.Window
java.awt.Frame
java.awt.Dialog
java.awt.Panel
java.applet.Applet
java.awt.ScrollPane
java.awt.Canvas

- **Component**: Base class for all UI elements (Button, Label, TextField, etc.).
  - Display of graphic object on the screen
  - Handles various keyboard and mouse events
- **Container**: Holds components (Panel, Frame, Applet, etc.).
- **Window**: A top-level container like Frame and Dialog.
- **Panel**: A generic container inside a Frame or Window.
- **Applet**: Used for embedding UI in web applications.

SWING

- Swing is a more advanced GUI toolkit than AWT, introduced in **Java 2 (JDK 1.2)**. It provides a richer set of UI components and greater flexibility.

Key Features of Swing

- **Lightweight**: Does not rely on the OS's native UI components.
  - Always generate similar type of output irrespective of the underlying platform
- **Pluggable Look and Feel**: Can be customized with themes.
- **More Components**: Provides advanced UI elements like JTable, JTree, JTabbedPane, etc.
- **Event-Driven**: Uses listeners to handle user interactions.

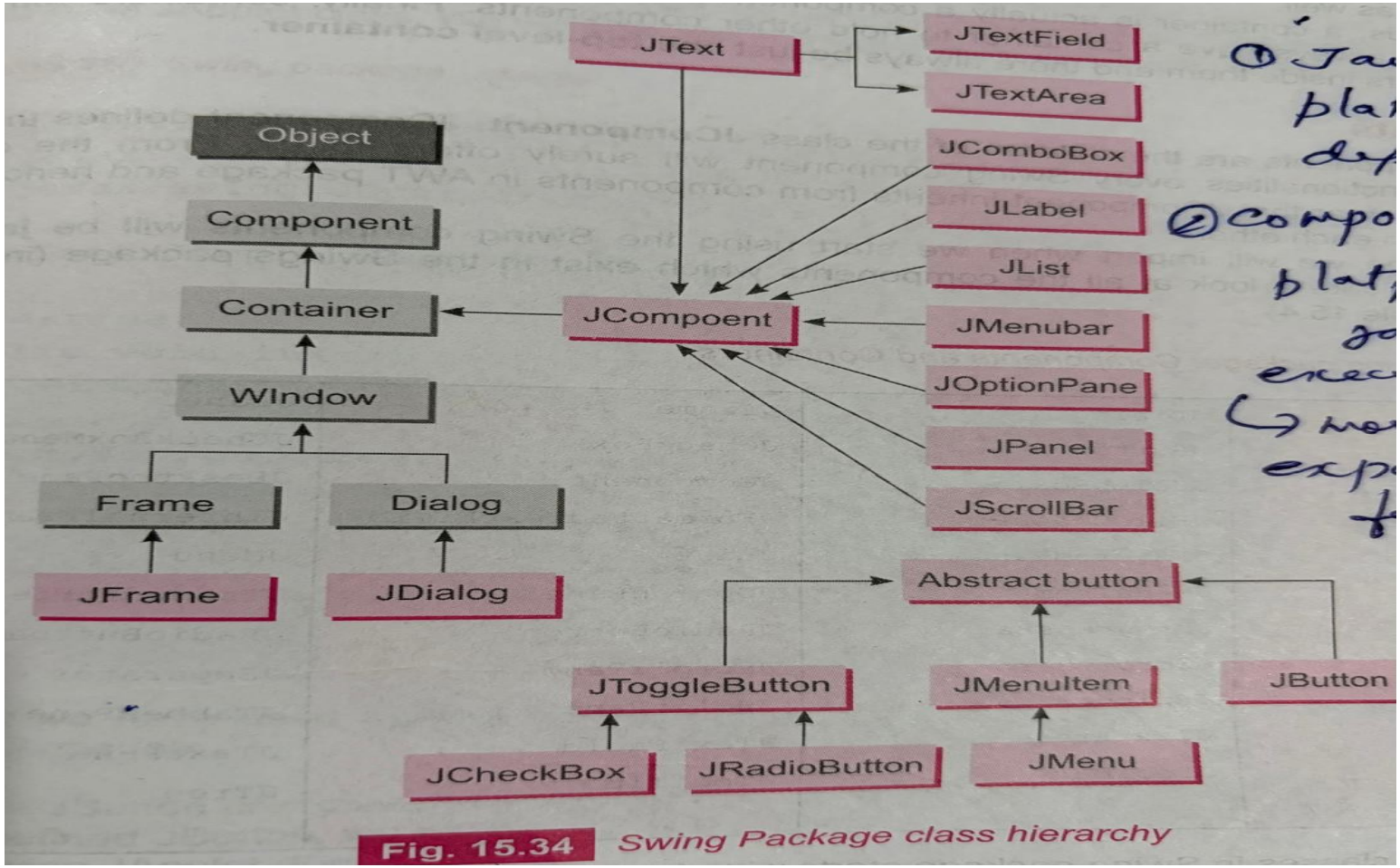


Fig. 15.34 *Swing Package class hierarchy*

In Java GUI, components are of two types:

1. **Heavyweight components (AWT)**
   - Use **native OS resources**
   - Example: Buttons, TextFields in AWT are created using the OS (Windows, Linux, etc.)
   - Each component has its own **peer** (a native screen resource)
2. **Lightweight components (Swing)**
   - Written **entirely in Java**
   - **Do not use native OS components**
   - Drawn using Java code on a common container (like JFrame)
   - Only the top-level container (like JFrame) is heavyweight; all others are lightweight

SWING vs AWT

Features	AWT	Swing
Dependency	Uses native OS components	Lightweight, written in pure Java
Performance	Slower (relies on OS)	Faster (handles rendering internally)
Look & Feel	OS-dependent	Can be customized
Components	Basic UI elements	Advanced UI elements (tables, trees, etc.)
Extensibility	Limited	Highly extensible
Thread Safety	Not thread-safe	Partially thread-safe (uses Event Dispatch Thread)

JFrame (Top-Level Container)

- JFrame is a **standalone, top-level window** with a title bar, minimize, maximize, and close buttons.
- It is commonly used for GUI applications.
- You can add components to a JFrame using its contentPane.

JApplet

- JApplet is a **Swing-based Applet** used for embedding Java applications in web browsers.
- Since applets are now deprecated, this class is rarely used.
- Similar to JFrame, but meant for use within a browser.

JPanel (Lightweight Container)

JPanel (Lightweight Container)

- JPanel is a **general-purpose container** that can group multiple components together.
- Unlike JFrame, JPanel does **not** have its own window; it must be added to another container.
- Used for better organization of GUI components.

Feature	JFrame	JPanel
Purpose	Acts as the **main window** of an application	Acts as a **container inside a window**
Type	Top-level container	Lightweight component
Execution	Can run independently	Cannot run independently
Package	javax.swing	javax.swing
Inheritance	Extends Frame (AWT)	Extends JComponent
Usage	Used to create GUI window	Used to organize/group components
Default Layout	BorderLayout	FlowLayout
Visibility	Needs setVisible(true)	Visible when added to a container

JButton

- **JButton** is a Swing component used to create a **clickable button** in a GUI application. It is part of the javax.swing package.

Key Features

- Displays **text, icon, or both**
- Generates an **event when clicked**
- Supports **ActionListener** for handling events
- Fully **customizable** (color, font, size,

- JButton b1 = new JButton(); // empty button
  JButton b2 = new JButton("Click"); // button with text
- JButton b3 = new JButton(new ImageIcon("img.png")); // button with image

What is ActionListener?

- ActionListener is an **interface** used to handle **button click events** (and similar actions).
- When a user clicks a button, an **event is generated**, and ActionListener handles it.

Step 1: Import package

- import java.awt.event.*;

Step 2: Registering the listener

```
class MyClass implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Clicked");
    }}

// In main:
btn.addActionListener(new MyClass());//Registering the Listener
```

addActionListener()

- Method used to **attach event handling** to a component

```
JButton btn = new JButton("Click");
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked");
    }
});
```

new ActionListener()

- Creating an **anonymous class** that implements ActionListener
- public void actionPerformed(ActionEvent e)

This method:

- is **mandatory** (must be overridden)
- Executes when the event occurs (e.g., button click)

```
JButton btn = new JButton("Click");


btn.addActionListener(new
ActionListener() {
   public void
actionPerformed(ActionEvent e) {
    System.out.println("Button Clicked");
    }
});
```

JLabel

- **JLabel** is a Swing component used to **display text, images, or both** in a GUI. It is mainly used for **showing information** to the user (not for input).
- JLabel is a **non-editable display component** in the javax.swing package.
- JLabel l1 = new JLabel(); // empty label
- JLabel l2 = new JLabel("Hello"); // label with text
- JLabel l3 = new JLabel(new ImageIcon("img.png")); // label with image

JTextField in Java

- JTextField is a part of the **Swing** package in Java (javax.swing.JTextField) used to create a single-line text input field.
- It allows users to enter and edit a string of text. It's commonly used in forms or GUI applications where user input is required.

Key Features:

- Allows single-line text input
- Can set or get text using setText() and getText() methods
- You can set the number of columns (visible width) when creating it
- Supports action listeners for handling user actions like pressing Enter

JTextArea in Java Swing

- JTextArea is a **multi-line text component** used to display or allow the user to enter multiple lines of text in a GUI application.

Key Features

- Allows **multiple lines of text**
- Supports **editing (typing, deleting)**
- Can be used for **displaying output**
- Often used with **scroll bars**

```
import javax.swing.*;

public class JTextAreaExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JTextArea Example");

        JTextArea ta = new JTextArea(5, 20);
        ta.setText("This is a JTextArea\nYou can write multiple lines.");

        frame.add(ta);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Use of JScrollPane in Java

- **Use of JScrollPane in Java**
- In **Swing**, JScrollPane is used to add **scrolling capability** to components that might exceed the visible area, like JTextArea, JTable, JList, etc. It automatically adds vertical and/or horizontal scrollbars when needed.
- **Why use JScrollPane?**
- Some components, like JTextArea, can contain more content than the space available in the window. Wrapping them inside a JScrollPane ensures the user can scroll through all the content.

JCheckBox (Check Box)

- JCheckBox allows the user to **select or deselect** an option.
- Multiple checkboxes can be selected at the same time.
- It's used when you want to offer **multiple selections** (e.g., selecting hobbies, skills, etc.).
- JCheckBox cb1 = new JCheckBox("Java");
- JCheckBox cb2 = new CheckBox("Python");
- JCheckBox cb3 = new JCheckBox("Python", true);

- cb1.setText("Java Programming");
- String value = cb1.getText();
- Sets the checkbox state.
- cb1.setSelected(true); // Checked

```
if(cb1.isSelected()) {
•       System.out.println("Java selected");
}
```

addActionListener(ActionListener a)

- Adds an action listener to handle events.

JRadioButton

- JRadioButton allows the user to **select only one option** from a group. To enforce this, you use a ButtonGroup to group related radio buttons together. Selecting one will deselect the others in the group.
- JRadioButton rb1 = new JRadioButton("Male");
- JRadioButton rb2 = new JRadioButton("Female");
- ButtonGroup group = new ButtonGroup();
- group.add(rb1);
- group.add(rb2);

Common example:

- Gender: Male / Female
- Payment: Cash / Card / UPI

Constructors

- JRadioButton()
- JRadioButton(String text)
- JRadioButton(String text, boolean selected)
- JRadioButton r1 = new JRadioButton("Male");
- JRadioButton r2 = new JRadioButton("Female", true);

Layout Manager

- In **Java Swing**, layout managers control how components (buttons, labels, text fields, etc.) are arranged within a **container** like a JFrame or JPanel.
- Instead of manually setting size and position, layout managers **automatically arrange components** based on rules.

FlowLayout (default for JPanel)

- **Components are arranged in a row**, like text in a paragraph.
- When the row is full, it wraps to the next line.
- You can set alignment (LEFT, CENTER, RIGHT).
- frame.setLayout(new FlowLayout());

- FlowLayout()
- FlowLayout(int align)
- FlowLayout(int align, int hgap, int vgap)

GridLayout

- Divides the container into a **grid of equal-sized cells** (rows x columns).
- Components are added row by row, left to right.
- GridLayout(int rows, int cols)
- GridLayout(int rows, int cols, int hgap, int vgap)

```
frame.setLayout(new GridLayout(2, 2)); // 2 rows, 2 columns
frame.add(new JButton("Button 1"));
frame.add(new JButton("Button 2"));
frame.add(new JButton("Button 3"));
frame.add(new JButton("Button 4"));
```

BorderLayout (default for JFrame)

- Divides the container into **five regions**:
  NORTH, SOUTH, EAST, WEST, CENTER.
- You can add one component to each region.

```
frame.setLayout(new BorderLayout());
frame.add(new JButton("Top"),
BorderLayout.NORTH);
frame.add(new JButton("Bottom"),
BorderLayout.SOUTH);
frame.add(new JButton("Left"),
BorderLayout.WEST);
frame.add(new JButton("Right"),
BorderLayout.EAST);
frame.add(new JButton("Center"),
BorderLayout.CENTER);
```